

Predicting Supreme Court Case Outcomes with a Bayesian Network: A Baseline-Aware Evaluation

Matthew Park

July 28, 2026

Abstract

The Supreme Court is one of the most powerful bodies in the United States government, and knowing which way it would lean on a particular case has obvious value. This paper builds a Bayesian network over the Supreme Court Database and asks whether case outcomes can be predicted from case attributes. The answer depends entirely on *which* attributes are permitted. Using all observable variables — several of which are recorded from the decision itself — the network reaches 81.7% Top-3 accuracy, comfortably above a no-features baseline at 76.1%. Restricted to information available *before* the Court rules, it reaches 76.1% Top-3: identical to the baseline that ignores every feature, and slightly worse on log-loss. We report proper scoring rules with bootstrap confidence intervals against four reference models, diagnose the structural reason the pre-decision model fails, and show that a BIC-scored structure search beats the hand-drawn graph on held-out log-likelihood. The negative result is the substantive one: on this feature set, most apparent predictive power is base rate.

1 Introduction

An earlier version of this work reported 73% Top-3 accuracy on 30 test cases and concluded that the network offered “a clear increase in accuracy than one could give on their own intuition.” Both halves of that claim turned out to be unsupported. A 30-case test set carries a bootstrap confidence interval of roughly ± 16 percentage points, and — more importantly — no baseline was computed, so there was nothing to compare the number against.

The Court’s dispositions are severely imbalanced. Across the full dataset, *affirmed* occurs in 30.1% of cases, *reversed and remanded* in 27.5%, and *reversed* in 22.0%. Those three outcomes alone account for 79.6% of all cases. A model that ignores every feature and names those three every single time therefore scores about 80% Top-3 — above the figure the original write-up presented as evidence that the model worked.

This paper rebuilds the model and, more consequentially, rebuilds its evaluation. The contributions are:

1. **Two evidence policies over one graph** (Section 3.1), separating a descriptive model that may use post-decision variables from a genuine forecast that may not.
2. **Exact inference by variable elimination** (Section 5), which both replaces sampling as the default and serves as ground truth for validating the three sampling engines.
3. **Hierarchical backoff CPTs** (Section 4), addressing the finding that 57.2% of the parent configurations this network requires were never observed during training.

Table 1: SCDB disposition codes, the 11-class prediction target.

Code	Disposition	Code	Disposition
1	Stay, petition, or motion granted	7	Affirmed and reversed in part and remanded
2	Affirmed (includes modified)	8	Vacated
3	Reversed	9	Petition denied or appeal dismissed
4	Reversed and remanded	10	Certification to or from a lower court
5	Vacated and remanded	11	No disposition
6	Affirmed and reversed in part		

4. **Baseline-anchored evaluation** (Section 7) with proper scoring rules and bootstrap confidence intervals against four reference models.
5. **A structural diagnosis** (Section 8) of *why* the pre-decision model cannot beat its baseline, and a structure search (Section 9) showing the hand-drawn graph is not the best available.

2 Data

We use the Supreme Court Database (SCDB) 2024_01 case-centered release [1], comprising 9,277 cases. After validation and removal of rows lacking a recorded disposition, 9,141 cases are retained. We split **chronologically**: the 7,312 earliest cases form the training set and the 1,829 most recent form the test set. A chronological split matters here because a random split would let the model train on future cases to predict past ones, which no deployed forecaster could do.

A correction to the variable set. SCDB ships a column named `splitVote`, which reads naturally as “the justices were divided.” It is not. It is an internal record-keeping flag marking cases whose votes span multiple records, and it is **constant (= 1) across all 9,277 rows**. A node fed by a constant carries no information, and the earlier model included one. The pipeline now derives

$$\text{voteSplit} = (\text{minVotes} > 0),$$

which splits 5,733 / 3,544 and does carry signal.

The 11 disposition codes are listed in Table 1.

3 Network Structure

The network is a four-layer directed acyclic graph over ten variables (Figure 1). Layer 1 carries court-level context; Layer 2 holds case properties as root nodes; Layer 3 holds intermediate nodes that mediate between case properties and the outcome; Layer 4 is the prediction target.

3.1 Two evidence policies over one graph

The same DAG supports two different experiments, distinguished only by which variables are supplied as evidence at prediction time. The graph and the CPTs are identical in both.

explanatory All nine observable variables. Several of these — `decision_type`, `split_vote`, `unconstitutional`, `precedent_alteration`, `case_supplement` — are recorded *from* the decision and are simply not available beforehand. This track is a description of how case attributes co-occur with outcomes. It is not a forecast.

Court information

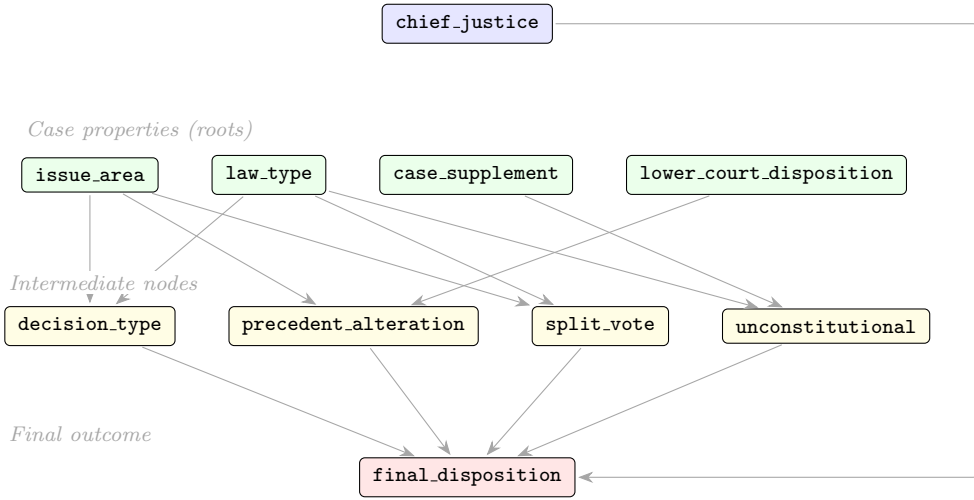


Figure 1: The hand-crafted network: 10 variables, 13 edges. Shading marks the four layers. Note that every Layer-2 variable except `chief_justice` reaches the outcome *only* through the intermediate layer — the fact that drives the result in Section 8.

ex_ante Only the four variables knowable before the Court deliberates: `chief_justice`, `issue_area`, `law_type`, and `lower_court_disposition`. The intermediate nodes become unobserved, so the posterior must marginalise over them.

Keeping both tracks in the same codebase is deliberate. The gap between them is the honest measure of how much of the explanatory track’s performance comes from peeking at the answer.

4 Conditional Probability Tables

Root nodes use a Laplace-smoothed marginal with strength α :

$$P(X = x) = \frac{N(X = x) + \alpha}{N + \alpha|X|}.$$

Child nodes are the harder case. The naive estimator,

$$P(X = x \mid \text{Pa}(X) = \mathbf{u}) = \frac{N(X = x, \text{Pa}(X) = \mathbf{u})}{N(\text{Pa}(X) = \mathbf{u})},$$

is undefined whenever the parent configuration $\mathbf{u}$ never occurs in training. This is not a rare edge case: `final_disposition` has five parents, and **57.2 % of the parent configurations the network needs were never observed**. The earlier implementation answered every one of them with a *uniform* distribution — the least informative guess available — and did so silently.

We instead use **hierarchical backoff** via Jelinek–Mercer interpolation. Writing $\mathbf{u}_k$ for the first k parents in a fixed ordering:

$$P_k(x \mid \mathbf{u}_k) = \lambda_k P_{\text{ML}}(x \mid \mathbf{u}_k) + (1 - \lambda_k) P_{k-1}(x \mid \mathbf{u}_{k-1}), \quad \lambda_k = \frac{N(\mathbf{u}_k)}{N(\mathbf{u}_k) + m}, \quad (1)$$

with P_0 the smoothed marginal. A configuration observed far more than m times is trusted almost entirely; a rare one is pulled toward an estimate conditioning on fewer parents. Crucially, an

Algorithm 1 Rejection sampling (retained for comparison)

Require: CPTs, observations $\mathbf{e}$, target value y , sample budget N

Ensure: Estimate of $P(\text{final_disposition} = y \mid \mathbf{e})$

```
1:  $hits \leftarrow 0$ ;  $accepted \leftarrow 0$ 
2: for  $i = 1$  to  $N$  do
3:    $s \leftarrow \text{ANCESTRALSAMPLE}(\text{CPTs})$ 
4:   if  $s$  agrees with  $\mathbf{e}$  on every observed variable then
5:      $accepted \leftarrow accepted + 1$ 
6:     if  $s[\text{final\_disposition}] = y$  then  $hits \leftarrow hits + 1$ 
7:     end if
8:   end if
9: end for
10: return  $accepted = 0 ? 0 : hits/accepted$ 
```

Table 2: Inference engines on the explanatory track (200 test cases).

Engine	Top-3	Time	TV distance from exact
Exact (variable elimination)	87.0%	0.02 s	— (reference)
Rejection sampling	55.5%	28.56 s	0.5477
Likelihood weighting	87.0%	8.27 s	0.0257
Gibbs sampling (MCMC)	87.0%	13.38 s	0.0232

unobserved configuration now falls back to a genuinely informative lower-order estimate rather than to uniform. The realised backoff rate is reported on every run rather than left implicit.

5 Inference

Four engines sit behind a single `query` interface.

5.1 Exact inference

Variable elimination computes the posterior exactly by summing out non-evidence, non-query variables in turn:

$$P(\text{final_disposition} \mid \mathbf{e}) \propto \sum_{\mathbf{h}} \prod_i P(X_i \mid \text{Pa}(X_i)),$$

where $\mathbf{h}$ ranges over hidden variables. With ten variables and modest cardinalities this is trivially cheap, and it is exact.

5.2 Sampling engines and their validation

The three sampling engines — rejection sampling, likelihood weighting, and Gibbs (MCMC) — are retained, but their role has changed. Previously they could only be compared to *one another*, which cannot distinguish “all three are correct” from “all three share a bug.” With an exact posterior available, each can be validated against a known answer.

Table 2 gives accuracy, wall-clock time, and total-variation distance from the exact posterior on the explanatory track.

Table 3: Total-variation distance from the exact posterior versus sample budget. Likelihood weighting and Gibbs reproduce exactly on every run; the rejection sampling row is one observed run of many possible ones (see text).

Engine	100	500	1000	5000
Rejection sampling [†]	0.5486	0.2546	0.1514	0.2276
Likelihood weighting	0.1134	0.0354	0.0164	0.0132
Gibbs sampling	0.1045	0.0376	0.0226	0.0136

Rejection sampling’s collapse is the textbook failure mode, now measured rather than asserted: with nine evidence variables almost every sample is discarded, leaving the estimate both wrong (TV 0.55) and $1400\times$ slower than exact. Table 3 shows the two weighted samplers converging toward the exact posterior as the budget grows. Rejection sampling does not converge cleanly, and its row is not reproducible between runs.

[†] **On the irreproducibility of the rejection row.** Repeated invocations at a fixed seed have produced 0.5486/0.8192/0.8192/0.1514 and 0.5486/0.5486/0.9087/0.2269 alongside the row above, and the sequence is not monotone in the sample budget. The cause is not sampling noise in the ordinary sense. `CPTBuilder.get_values` returns a `set`, so nodes taking string values iterate in an order that Python’s per-process hash randomisation varies from run to run; the index a given random draw selects therefore maps to a different value each time. Likelihood weighting and Gibbs are insensitive to this because they aggregate over every value of a node. Rejection sampling, accepting on the order of one sample in a hundred under nine evidence variables, rests on so few accepted draws that the ordering determines the estimate. Sorting the value sets would restore determinism at the cost of remapping every sampler’s draws, and with them the figures reported elsewhere in this paper; the instability is recorded here rather than papered over.

Exact inference is also the right default for a further reason: when all of a leaf’s parents are observed, the posterior is a single CPT row. The sampling engines were spending 1,000 draws to approximate a dictionary lookup.

6 Evaluation Methodology

Reference models. Four baselines implement the same `query` interface as the network and run through the identical evaluation path: a **marginal** model that ignores all features and returns the training base rate; a **majority-class** model; **multinomial logistic regression**; and **gradient boosting**. The marginal model is the one that matters — it is the floor any claim of predictive power must clear.

Metrics. We report Top- k accuracy for $k \in \{1, 3, 5\}$ alongside **proper scoring rules**: log-loss, Brier score, expected calibration error, and macro one-vs-rest AUC. Proper scoring rules cannot be gamed by predicting the marginal distribution, which on this task is precisely the failure mode worth guarding against. Every point estimate carries a bootstrap 95% confidence interval over 500 resamples.

Table 4: Multi-class disposition (11 classes), **explanatory** track — all observable variables, including several recorded from the decision. Brackets are bootstrap 95 % CIs.

Model	Top-1	Top-3	Top-5	Log-loss	Brier	ECE	AUC
Marginal (no features)	26.7 [24.4, 28.6]	76.1 [74.5, 77.9]	97.1	1.6244	0.7777	0.042	0.500
Majority class	26.7 [24.4, 28.6]	39.3 [37.1, 41.6]	94.8	5.0015	1.4502	0.723	0.500
Logistic regression	34.2 [32.0, 36.3]	81.4 [79.7, 83.0]	98.3	1.5302	0.7474	0.044	0.718
Gradient boosting	33.6 [31.6, 35.5]	80.5 [78.8, 82.1]	97.9	1.6187	0.7593	0.095	0.658
Bayesian network	32.6 [30.4, 34.5]	81.7 [80.0, 83.4]	98.0	1.5920	0.7495	0.050	0.553

Table 5: Multi-class disposition (11 classes), **ex-ante** track — only information knowable before the Court rules.

Model	Top-1	Top-3	Top-5	Log-loss	Brier	ECE	AUC
Marginal (no features)	26.7 [24.4, 28.6]	76.1 [74.5, 77.9]	97.1	1.6244	0.7777	0.042	0.500
Logistic regression	29.4 [27.3, 31.2]	77.9 [76.2, 79.6]	97.9	1.6116	0.7749	0.077	0.673
Gradient boosting	30.2 [28.2, 32.5]	78.5 [76.8, 80.0]	97.2	1.6901	0.7822	0.090	0.659
Bayesian network	27.0 [24.7, 29.0]	76.1 [74.5, 77.9]	97.8	1.6339	0.7764	0.064	0.520

7 Results

Table 4 shows the network at 81.7% Top-3, above the no-features baseline at 76.1% with non-overlapping confidence intervals. That is a real effect. It is also not a forecast: five of the nine evidence variables are recorded from the decision being predicted.

Table 5 is the honest headline. Restricted to pre-decision information, the network scores 76.1% Top-3 — *identical* to the model that ignores every feature — and is slightly *worse* on log-loss (1.6339 versus 1.6244). The two tree-based reference models do marginally better, but neither clears its baseline by more than the interval width.

7.1 Binary affirm / reverse

Table 6 reports the standard formulation in the literature: affirm = disposition 2, reverse/vacate = dispositions 3, 4, 5, 8, other dispositions excluded ($n = 1,729$).

Katz, Bommarito & Blackman [2] report roughly 70.2% case-level accuracy on this task using time-evolving random forests over a much larger feature set and a different evaluation protocol; the numbers are not directly comparable. The more useful observation is that *always predicting “reversed”* scores 71.8% on this test set, because the Court reverses far more often than it affirms. Any binary accuracy figure in this literature — including ours — should be read against that floor.

7.2 What the AUC column says

AUC exceeds 0.5 everywhere, including the ex-ante track (0.520–0.673). There *is* signal in pre-decision variables: the models rank cases better than chance. It is simply not enough to change which class ends up on top, because one class dominates. Reporting accuracy alone would conceal this; reporting AUC and log-loss alongside it does not.

Table 6: Binary affirm / reverse task ($n = 1,729$).

Model	Track	Accuracy	Log-loss	AUC
Marginal (no features)	—	71.8 [69.6, 73.7]	0.6015	0.500
Bayesian network	explanatory	72.6 [70.4, 74.7]	0.5703	0.599
Logistic regression	explanatory	72.1 [69.7, 74.1]	0.5644	0.663
Bayesian network	ex_ante	71.8 [69.6, 73.7]	0.6036	0.561
Gradient boosting	ex_ante	69.7 [67.5, 71.7]	0.6042	0.572

Table 7: Paths from ex-ante variables to the outcome. “Direct parent?” means a direct parent of `final_disposition` in the graph.

Ex-ante variable	Direct parent?	Reaches the outcome via
<code>chief_justice</code>	yes	—
<code>issue_area</code>	no	<code>decision_type</code> , <code>precedent_alteration</code> , <code>split_vote</code>
<code>law_type</code>	no	<code>decision_type</code> , <code>split_vote</code> , <code>unconstitutional</code>
<code>lower_court_disposition</code>	no	<code>precedent_alteration</code> only

8 Why the Ex-Ante Model Fails: A Structural Diagnosis

The ex-ante result is not a bug, and the graph explains it precisely. Of the four pre-decision variables, only `chief_justice` is a direct parent of the outcome (Table 7).

Every other ex-ante variable reaches `final_disposition` *only* through the four intermediate nodes — and all four are post-decision. When they are unobserved, they act as a bottleneck that washes the signal out.

`lower_court_disposition` is the sharpest case. Whether the court below affirmed or reversed is plausibly the single most informative thing knowable before the ruling, and it is genuinely associated with the outcome in the data ($G = 352.4$, $df = 99$, $p \approx 4 \times 10^{-30}$). But its only path to the outcome runs through `precedent_alteration`, a near-constant binary. Conditioning on it moves the posterior by a total-variation distance of approximately 0.001. *The information is present in the data and the graph cannot carry it.*

This is pinned by a regression test, so that anyone who later adds a direct edge will see the expectation fail and learn why it was there.

9 Hand-Crafted versus Learned Structure

If the intermediate layer is the problem, the natural question is whether the data supports it at all. We ran BIC-scored hill-climbing from the empty graph with add, remove, and reverse moves, and an in-degree cap of four (Table 8).

The learned graph wins on BIC — which it was optimising, so that is expected — but it also wins on **held-out log-likelihood**, which it was not. The two graphs share only **3 of 13 edges**. The search adds `lawType` $\rightarrow$ `caseDisposition` and drops most of the hand-specified parents of the outcome, including `decisionType`, `chief`, `precedentAlteration`, and `declarationUncon`.

Read plainly: on this dataset the domain-informed structure is worse than one derived from the data, and the intermediate-node layer is largely not earning its place. This corroborates Section 8 from an independent direction.

Table 8: Hand-crafted versus learned network structure.

	BIC	Held-out log-likelihood	Edges
Hand-crafted (domain)	-82,533.75	-15,088.07	13
Learned (BIC search)	-56,830.93	-14,315.70	11

We also replaced raw mutual information with **normalised** mutual information and a G -test with proper degrees of freedom, since raw MI’s cardinality bias systematically favours high-arity variables in exactly the pairwise screening the original structure was built from.

10 Limitations

- **The ex-ante feature set is thin.** SCDB carries other genuinely pre-decision fields — `certReason`, `jurisdiction`, `lcDisagreement`, `caseSource` — that are absent from the graph and would be the natural next extension. The negative result in Table 5 bounds *this* feature set, not pre-decision prediction in general.
- **The intermediate layer is unsupported.** Structure search does not recover it, and the outcome node’s five-parent family is sparse enough that 57.2% of its configurations require backoff.
- **Case-level modelling only.** The literature’s stronger results come from justice-level vote prediction aggregated to a case outcome, which SCDB’s justice-centered release supports and this work does not use.

11 Conclusion

A Bayesian network can describe the Supreme Court’s record well: given all observable case attributes, it reaches 81.7% Top-3 accuracy, significantly above a no-features baseline. But most of those attributes are recorded from the decision. Restricted to what is knowable beforehand, the same network matches the baseline that ignores every feature and loses to it on log-loss — and the graph structure explains precisely why, with the most informative pre-decision variable bottlenecked behind a near-constant intermediate node.

The methodological point generalises beyond this dataset. On a task with an 80% base rate concentrated in three classes, a Top-3 accuracy figure reported without a baseline is uninterpretable, and one computed on 30 test cases is noise. The first version of this work reported exactly that. The apparatus here — baselines sharing the evaluation path, proper scoring rules, bootstrap intervals, and an exact engine to validate the approximate ones — exists to make that particular mistake hard to repeat.

References

- [1] Washington University Law. (2024). *The Supreme Court Database*, 2024_01 case-centered release. <http://scdb.wustl.edu>
- [2] Katz, D. M., Bommarito, M. J., & Blackman, J. (2017). A general approach for predicting the behavior of the Supreme Court of the United States. *PLoS ONE*, 12(4), e0174698.

- [3] Russell, S., & Norvig, P. *Artificial Intelligence: A Modern Approach*, Ch. 14: Probabilistic reasoning and sampling methods for Bayesian networks.
- [4] Koller, D., & Friedman, N. (2009). *Probabilistic Graphical Models: Principles and Techniques*. MIT Press.